

Cell-Internal vs. Substrate State: The Architectural Distinction That Makes the Substrate-Cell Boundary Operational

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: 2 May 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the architectural distinction between cell-internal state and substrate state — the line on which the source paper's substrate-cell boundary, source-of-truth commitment, and linear-cost commitment all operationally rest — so that downstream work can locate any specific piece of state on the correct side of the line by architectural test rather than by intuition.

Abstract

The Coordination Knowledge Substrate (CKS) pattern is built on a two-layer separation between substrate state, which persists across executions and is authoritative for coordination questions, and cell-internal state, which exists only during a single cell execution and is released when the execution completes. The parent foundational note formalizes the boundary between the two layers; sibling decomposition notes formalize what each layer commits to alone and what crosses the boundary in each direction. None of these formalizes the line that determines, for any particular piece of state, which side it belongs on. This note formalizes that line. It states the four jointly required properties that distinguish substrate state from cell-internal state (persistence, addressability, authority, provenance), enumerates six categories of cell-internal state that recur in implementations, distinguishes the architectural distinction from four adjacent technical patterns commonly conflated with it (database vs. cache, persistent vs. transient memory, hot vs. cold storage, working vs. long-term memory), names six failure modes that violate the distinction, and provides an operational test for placing any specific piece of state on the correct side. The distinction is architectural, not implementational: state belongs on a side based on the coordination role it plays, not on the storage medium that holds it.

1. Why this distinction needs to be formalized as standalone

The CKS pattern's substrate-cell boundary commits the substrate layer to persistence and the cell layer to bounded execution that does not carry state across invocations (§2.1, §4.1). Sibling decomposition notes formalize each layer's standalone commitments and the boundary-crossing operations in each direction. The substrate's persistence commitment says that state living in the substrate persists; it does not say what state belongs there. The cell's statelessness commitment says that cell-internal state does not survive an execution; it does not say what state qualifies as cell-internal. The line is what this note formalizes.

The motivating cases are pieces of state that could plausibly live on either side. An LLM's tokens during a cell's execution — its working context, the reasoning it performs, the intermediate outputs it produces — is cell-internal: scoped to the execution, released when the execution completes. A decision the cell records to the substrate as authoritative content is substrate state: persistent, addressable, governing what subsequent work treats as the case. A draft of a decision the cell considered but did not commit is cell-internal: unrecorded, ephemeral. The rationale attached to a committed decision is substrate state: part of the path-retraceability content the substrate carries by §3.1's commitment. The architecture must place each of these on the correct side with precision; if the line is fuzzy, implementations drift toward the failure pattern §6.2 of the source paper names *context rot* — coordination state migrating into LLM context where it is unaddressable and ungoverned.

A second motivation is the linear-cost commitment. Cells that accumulate per-execution state across executions break the cost model: cell execution cost begins to scale with the cell's history rather than with the cell's task scope. The distinction is what prevents that drift. Substrate state lives in the substrate, where its accumulation is fine because cells read selectively under orchestration rules; cell-internal state is bounded to the execution and cannot accumulate across executions because executions are independent.

A third motivation is governance. Substrate state is governed by humans through the three rights named in the human-governed commitment; cell-internal state is not, and should not be. If cell-internal state holds coordination content, that content has escaped governance — the authority architecture does not apply where the state lives. Naming the line is what keeps governance architecturally meaningful when implementations introduce intermediate layers — caches, agent memory, scratchpads — whose architectural role is not initially clear.

2. The four properties that distinguish substrate state from cell-internal state

A piece of state is **substrate state** in the architectural sense if and only if it has all four of the following properties.

(a) Persistence across executions. Substrate state persists from one cell execution to the next, across sessions, and across time, modulo deletions authorized under the modify right. Cell-internal state exists only during a single execution and is released when the execution completes. The persistence test is operational: state that affects the next execution because it persists is substrate state; state that does not is cell-internal.

(b) Addressability in inspectable form. Substrate state is addressable as a discrete unit that humans can read directly under the inspect right, in the form the substrate actually carries it. Cell-internal state need not be addressable; it is the working content of one execution and need not survive that execution as identifiable content. State that is addressable and inspectable is substrate state in the architectural sense; state that is not is cell-internal.

(c) Authority over coordination questions. Substrate state is what the system reads to answer the coordination questions: what was decided, by whom, under what authority, with what rationale, and what conflicts remain unresolved (§11.3). Cell-internal state has no authority over these questions; it is bounded to the execution that produced it. State that the system consults to answer coordination questions is substrate state by definition; state that does not affect such answers is cell-internal.

(d) Provenance attribution. Substrate state carries the six provenance fields the path-retraceability commitment requires: writer, timestamp, antecedent reference, rule reference, rationale where applicable, and relationship to contradicting content where applicable. Cell-internal state does not carry provenance; it is scoped to its execution and need not be retraceable as independent content. State that carries provenance is substrate state; state that does not is cell-internal.

The four properties are co-implicating. Substrate state has all four; cell-internal state has none. A piece of state with some properties but not others is **architecturally ambiguous**, and the ambiguity must be resolved by the deployment — either by committing the state to substrate (with all four properties) or by releasing it as cell-internal (with none). Architectural ambiguity left unresolved is the seedbed for the failure modes named in §6.

3. What counts as cell-internal state

Cell-internal state recurs in implementations in six categories. The list is not exhaustive, but it covers the cases that arise most often.

(a) LLM context during execution. When a cell uses an LLM as mediator, the LLM's context — the tokens it processes, the reasoning it performs, the intermediate outputs it produces — is cell-internal during the execution. Each execution starts with fresh context, populated by what the orchestration rule specifies and what the cell reads from the substrate. LLM context is the canonical example.

(b) Intermediate reasoning. Cells, whether LLM-mediated or deterministic, may produce intermediate computations, working calculations, or scratchpad content during execution. These are cell-internal. Only the cell's final outputs, written to the substrate as cell→substrate writes under orchestration rules, become substrate state.

(c) Unrecorded drafts. A cell may produce candidate outputs, consider alternatives, or generate intermediate writes that do not satisfy the orchestration rule's commit conditions. These drafts are cell-internal and are released when the execution completes. Only writes the rule authorizes for commit become substrate state.

(d) Execution-scoped caches. A cell may cache intermediate lookups, computed values, or derived data during execution. These caches are cell-internal — they exist to serve the execution and have no

role beyond it. A "cache" that persists across executions is something else (a derived view, with its own architectural commitments).

(e) Tool-call results not yet committed as writes. When a cell invokes an external tool — an API, a search service, an adjacent component — the result the tool returns is cell-internal until the cell decides, under orchestration rules, what part of the result to write to the substrate. The full result is cell-internal; the portion the cell writes (with provenance attribution to the tool source) becomes substrate state.

(f) Control-flow state. Loop counters, branch conditions, retry state, and other control-flow markers within an execution are cell-internal. They serve the execution and have no architectural role beyond it.

The principle these six instantiate: state that exists to serve a single execution and has no continuing coordination role is cell-internal. The principle is not "what should be remembered vs. forgotten" — cell-internal state is appropriately released when the execution completes, not lost; substrate state is retained because it is authoritative for coordination questions, not because the architecture privileges memory over forgetting.

4. What the distinction is NOT

Four adjacent technical patterns are commonly conflated with the cell-internal vs. substrate state distinction. Each names a real engineering distinction in some other dimension; none is the architectural distinction the CKS pattern commits to.

Not database vs. cache. Database vs. cache is a technical distinction about storage technology and access patterns: databases are typically durable and authoritative; caches are typically fast and ephemeral. The architectural distinction is orthogonal. Cell-internal state may be stored in any technology, including a database; substrate state may be stored in any technology, including an in-memory store, if the in-memory store is part of the substrate's architectural identity. The distinction is about coordination role, not storage technology.

Not persistent vs. transient memory. Persistent vs. transient is a technical distinction about whether memory survives process restart. The architectural distinction is about coordination scope: state that survives process restart but exists only to serve a single cell's execution across multiple invocations is still cell-internal in the architectural sense. The distinction is about coordination role, not memory durability.

Not hot vs. cold storage. Hot vs. cold is a technical distinction about access frequency and performance tier. The architectural distinction is again orthogonal: substrate state may be held in either tier, and so may cell-internal state. The distinction is about coordination authority, not access tier.

Not working memory vs. long-term memory. Working vs. long-term is a cognitive-science framing sometimes applied to AI agent architectures. The architectural distinction in CKS is not cognitive — it is about coordination state vs. execution state. A cell with rich "working memory" during execution is fine if all coordination-relevant content is committed to substrate under rule-authorized writes; a cell with sparse "working memory" but with state escaping to substrate without rule authorization is violating the distinction. The architecture is not about cognition; it is about where coordination state

lives.

5. Why this distinction is consequential downstream

The cell-internal vs. substrate state distinction is not its own primary commitment — it is the operational basis on which several other CKS commitments rest at the boundary. Four connections name what depends on it.

The **linear-cost commitment** depends on cells not accumulating per-substrate-element state across executions. The distinction is what prevents the accumulation: cell-internal state is bounded to its execution and released after; substrate state lives in the substrate, where cells read selectively. Without the distinction, cells drift toward stateful behavior across invocations and the cost model breaks.

The **source-of-truth commitment** depends on coordination state living in the substrate. The distinction is what enforces that location: state with the four substrate-state properties lives in the substrate; state without them does not affect coordination authority. Without the distinction, coordination state migrates into cell-internal locations and the substrate loses its authoritative status while still appearing nominally to hold one.

The **human-governed commitment** depends on coordination state being inspectable, modifiable, and overridable. The distinction is what places coordination state under governance: substrate state has the architectural rights; cell-internal state does not, and should not, because it is not coordination state. State that escapes the distinction escapes governance.

The **path-retraceability commitment** depends on coordination state carrying provenance. The distinction is what enforces provenance attribution: substrate state carries the six fields; cell-internal state does not need to, because it is not retraceable independent content. Without the distinction, provenance becomes optional and the retraceability commitment becomes unverifiable.

Each of these commitments has its own foundational note that does the load-bearing argumentation. The point here is narrower: each rests, at the layer where state is placed, on the line this note formalizes.

6. Failure modes that violate the distinction

A system can fail the cell-internal vs. substrate state distinction in six ways. Naming each is what allows downstream remediation.

(a) Coordination state in cell-internal storage. When state that should answer coordination questions lives in cell-internal locations — LLM context across turns, agent memory scoped to executions, in-memory state that is supposed to be released — the source-of-truth commitment is violated. The substrate is no longer authoritative because the authoritative state is elsewhere. This is the canonical failure mode; the source paper's §6.2 frames it as *context rot*, and the source-of-truth note treats it under the rubric of agent memory as source of truth.

(b) Cell-internal state accumulating across executions. When cells maintain state between their own invocations — caching across executions, remembering previous decisions in execution-scoped storage

that was supposed to be released — the cell becomes stateful in the architectural sense. The cell-statelessness commitment is violated, and the linear-cost property is compromised because cell execution cost begins to scale with the cell's history.

(c) Substrate state held in cell-internal storage. When state that has the four substrate-state properties is held in cell-internal locations rather than in the substrate, the substrate layer's identity is violated even though a substrate exists nominally. The state functions as substrate state but is not in the substrate; the architecture has the appearance of CKS-coherence without the substance.

(d) Cell-internal state escaping to substrate without authorization. When intermediate state, drafts, or working content from a cell's execution becomes substrate content without going through the cell→substrate write operation under orchestration rules, the boundary-crossing semantics are violated. Cell-internal state has appeared in substrate state without rule authorization or provenance; the substrate's content carries items it cannot account for.

(e) Substrate state mistaken for cell-internal state during release. When state with substrate-state properties is released at the end of a cell execution because the architecture conflated it with cell-internal state, the substrate has lost authoritative content. This typically happens when implementations identify state by its location (in-memory vs. on-disk) rather than by its architectural role.

(f) The "smart cache" failure. This pattern deserves a slightly fuller treatment because it is the most consequential drift in current AI infrastructure. An architecture introduces a layer between cells and substrate that holds state with some substrate-state properties — persistence, addressability — but not others — authority, provenance. The state is architecturally ambiguous: it persists across executions but lacks authority over coordination questions and carries no provenance. Coordination questions answered from this layer fail the source-of-truth commitment; questions answered from the actual substrate are correct but slower; the implementation drifts toward answering from the cache because it is faster, and the substrate becomes ceremonial — present in the architecture diagram, absent from the path coordination questions actually traverse. The remedy is to resolve the ambiguity: either commit the layer's content to the substrate with full substrate-state properties, or release it with each execution and treat it as a cell-internal performance optimization.

7. Operational test

A piece of state belongs to **substrate state** in the architectural sense if and only if all of the following are true:

- (a) The state persists across cell executions, sessions, and time, surviving any single cell's invocation, modulo deletions authorized under the modify right.
- (b) The state is addressable as a discrete unit that humans can inspect directly under the inspect right, in the form the substrate carries it.
- (c) The state is consulted to answer coordination questions: what was decided, by whom, under what authority, with what rationale, and what conflicts remain unresolved.

(d) The state carries provenance: writer, timestamp, antecedent reference, rule reference, rationale where applicable, and relationship to contradicting content where applicable.

A piece of state belongs to **cell-internal state** in the architectural sense if and only if all of the following are true:

(e) The state exists only during a single cell execution and is released when the execution completes.

(f) The state has no role in answering coordination questions beyond serving the execution that produced it.

(g) The state does not carry provenance as architecturally required content.

(h) The state does not affect subsequent cell executions except through substrate writes the orchestration rule authorizes.

A piece of state that satisfies some but not all of (a)–(d) and some but not all of (e)–(h) is architecturally ambiguous. The deployment must resolve the ambiguity by either committing the state to substrate (acquiring the full set of (a)–(d) properties) or releasing it as cell-internal (carrying the full set of (e)–(h) properties). Ambiguity left unresolved produces, over time, the failure modes named in §6.

8. Why naming this distinction matters

Implementations that conflate cell-internal state with substrate state — typically in pursuit of performance, agent-style "memory," or simplified architecture — produce systems where coordination state escapes governance, where the substrate becomes ceremonial rather than authoritative, and where the cost model breaks because cells accumulate state across executions. The drift toward this conflation is steady in implementations that begin without the architectural distinction in place; the substrate gradually loses authority to caches, agent memory, or "smart" intermediate layers, and the system retains the appearance of CKS-coherence while losing the substance.

Naming the distinction as a standalone architectural commitment — with the four properties of substrate state in §2, the six categories of cell-internal state in §3, and the four adjacent-pattern distinctions in §4 — gives downstream implementers a precise specification for placing any specific piece of state on the correct side of the line. The distinction is not optional architectural hygiene. It is the operational basis on which the linear-cost commitment, the source-of-truth commitment, the human-governed commitment, and the path-retraceability commitment all rest at the boundary.

Subsequent work that adopts the CKS pattern, extends it, or argues against it should use "substrate state" and "cell-internal state" in the senses formalized here. Subsequent work that uses either term in a different sense is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Cell-Internal vs. Substrate State: The Architectural Distinction That Makes the Substrate-Cell Boundary Operational*. 2 May 2026. ORCID: 0009-0004-8065-3235.